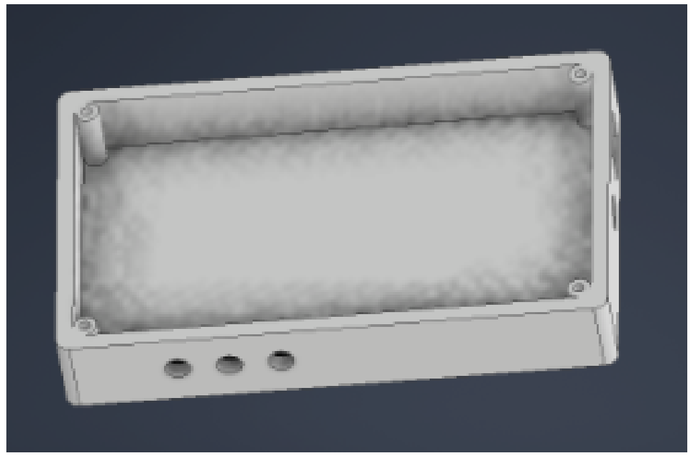
ENTREGABLE 10

GRUPO 6

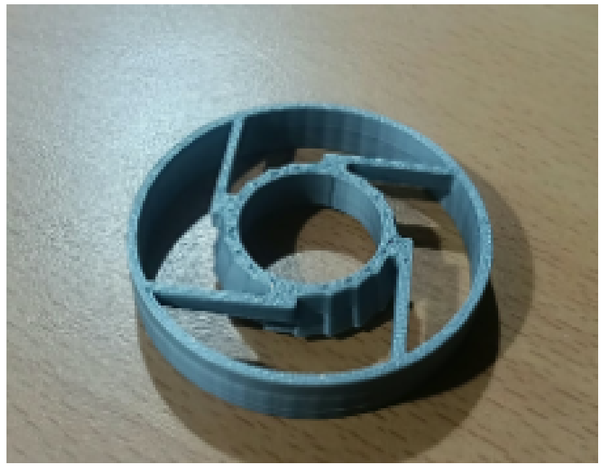
Mecánica y fabricación

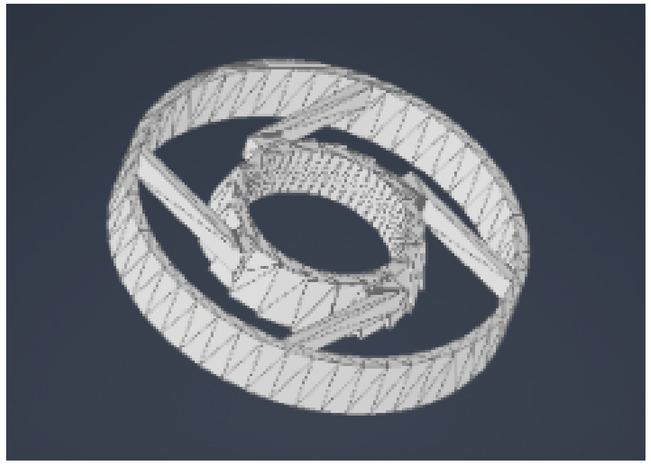




Caja de control: Se fabricó mediante impresión 3D para alojar y proteger los componentes electrónicos del sistema.

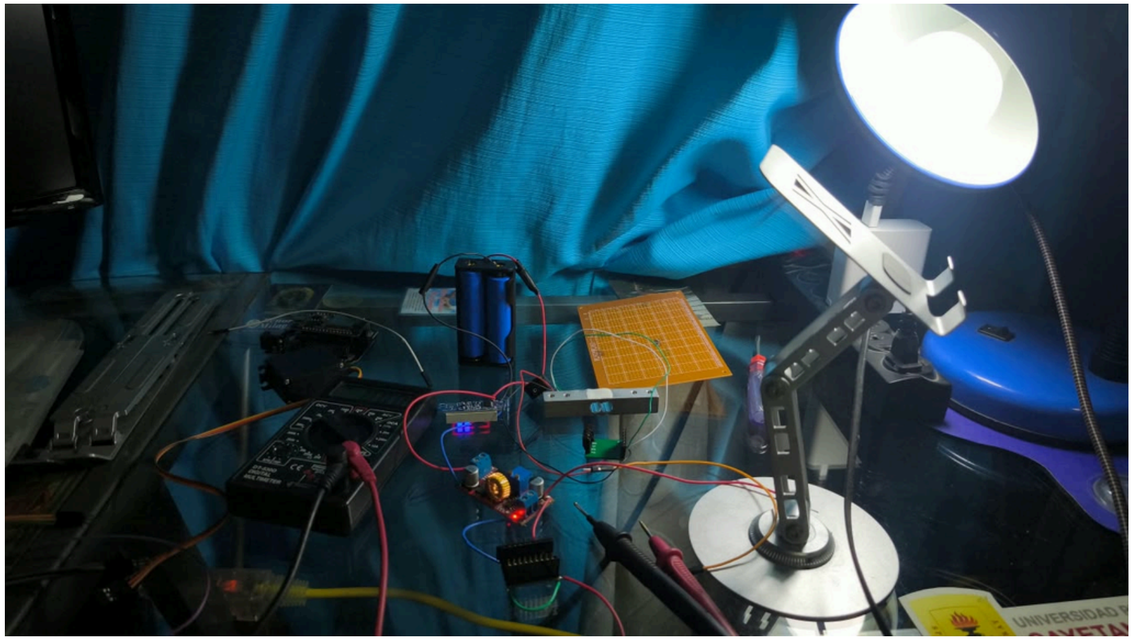
Mecánica y fabricación





Trinquete: Se fabricó mediante impresión 3D como parte del mecanismo de bloqueo, permitiendo restringir el movimiento en una dirección.

Electrónica y sensores



Montaje electrónico

Programación y control

```
96  void setup() {
97    Serial.begin(115200);
98    Wire.begin(PIN_I2C_SDA, PIN_I2C_SCL);
99
100   bascula.begin(PIN_HX711_DOUT, PIN_HX711_SCK);
101   bascula.set_scale(HX711_UNIDADES_POR_CUENTA);
102   if (bascula.is_ready()) {
103     Serial.println("HX711 listo. Retire toda carga: se tarará en 2 segundos.");
104     delay(2000);
105     bascula.tare(20);
106   } else {
107     Serial.println("ERROR: HX711 no responde; revisar DOUT, SCK y alimentación.");
108   }
109
110   if (!imu.begin(0x68, &Wire)) {
111     Serial.println("ERROR: MPU6050 no responde en 0x68 (pruebe 0x69 si AD0=HIGH).");
112   } else {
113     imu.setAccelerometerRange(MPU6050_RANGE_4_G);
114     imu.setGyroRange(MPU6050_RANGE_500_DEG);
115     imu.setFilterBandwidth(MPU6050_BAND_21_HZ);
116     Serial.println("MPU6050 listo. Mantenga la rodilla inmóvil para fijar reposo.");
117   }
118
119   servoRodilla.setPeriodHertz(50);
120   servoRodilla.attach(PIN_SERVO, 500, 2500);
121   servoRodilla.write((int)SERVO_CENTRO_GRADOS);
122   anguloServoActual = SERVO_CENTRO_GRADOS;
123
124   Serial.println("ESP32 listo y activo. El interruptor físico controla la alimentación.");
125   Serial.println("El nivel de batería se consulta visualmente en el indicador 2S.");
126 }
```

Inicialización del sistema: Al encender el dispositivo, se inicializan y calibran los componentes electrónicos, como el sensor MPU6050 y el módulo HX711, además de preparar el servomotor para su funcionamiento antes de ejecutar el programa principal.

Programación y control

```
63  float calcularInclinacionAccel(float ax, float ay, float az) {
64    // Ángulo del plano X-Z
65    return atan2f(ax, sqrtf(ay * ay + az * az)) * 180.0f / PI;
66  }
67
68  bool leerAngulo(float &angulo) {
69    sensors_event_t aceleracion, velocidadAngular, temperatura;
70    if (!imu.getEvent(&aceleracion, &velocidadAngular, &temperatura)) return false;
71
72    const uint32_t ahora = millis();
73    float dt = (ahora - ultimaLecturaMs) / 1000.0f;
74    if (dt <= 0.0f || dt > 0.2f) dt = INTERVALO_LECTURA_MS / 1000.0f;
75    ultimaLecturaMs = ahora;
76
77    const float anguloAccel = calcularInclinacionAccel(
78        aceleracion.acceleration.x,
79        aceleracion.acceleration.y,
80        aceleracion.acceleration.z);
81    const float gyroXGradosS = velocidadAngular.gyro.y * 180.0f / PI;
82
83    if (!referenciaLista) {
84      anguloFiltrado = anguloAccel;
85      anguloReposo = anguloAccel;
86      referenciaLista = true;
87    } else {
88      anguloFiltrado = CONSTANTE_FILTRO_COMPLEMENTARIO *
89                       (anguloFiltrado + gyroXGradosS * dt) +
90                       (1.0f - CONSTANTE_FILTRO_COMPLEMENTARIO) * anguloAccel;
91    }
92    angulo = anguloFiltrado - anguloReposo;
93    return true;
94  }
```

Funcionamiento del programa principal: El sistema realiza continuamente lecturas de la fuerza de pisada y del ángulo de inclinación de la rodilla. Cuando detecta apoyo y un ángulo menor a 0°, activa el servomotor para ajustar su posición. Al dejar de detectar apoyo, el servomotor regresa a su posición central, preparándose para el siguiente paso.

Programación y control

```
96  void setup() {
97    Serial.begin(115200);
98    Wire.begin(PIN_I2C_SDA, PIN_I2C_SCL);
99
100   bascula.begin(PIN_HX711_DOUT, PIN_HX711_SCK);
101   bascula.set_scale(HX711_UNIDADES_POR_CUENTA);
102   if (bascula.is_ready()) {
103     Serial.println("HX711 listo. Retire toda carga: se tarará en 2 segundos.");
104     delay(2000);
105     bascula.tare(20);
106   } else {
107     Serial.println("ERROR: HX711 no responde; revisar DOUT, SCK y alimentación.");
108   }
109
110   if (!imu.begin(0x68, &Wire)) {
111     Serial.println("ERROR: MPU6050 no responde en 0x68 (pruebe 0x69 si AD0=HIGH).");
112   } else {
113     imu.setAccelerometerRange(MPU6050_RANGE_4_G);
114     imu.setGyroRange(MPU6050_RANGE_500_DEG);
115     imu.setFilterBandwidth(MPU6050_BAND_21_HZ);
116     Serial.println("MPU6050 listo. Mantenga la rodilla inmóvil para fijar reposo.");
117   }
118
119   servoRodilla.setPeriodHertz(50);
120   servoRodilla.attach(PIN_SERVO, 500, 2500);
121   servoRodilla.write((int)SERVO_CENTRO_GRADOS);
122   anguloServoActual = SERVO_CENTRO_GRADOS;
123
124   Serial.println("ESP32 listo y activo. El interruptor físico controla la alimentación.");
125   Serial.println("El nivel de batería se consulta visualmente en el indicador 2S.");
126 }
```

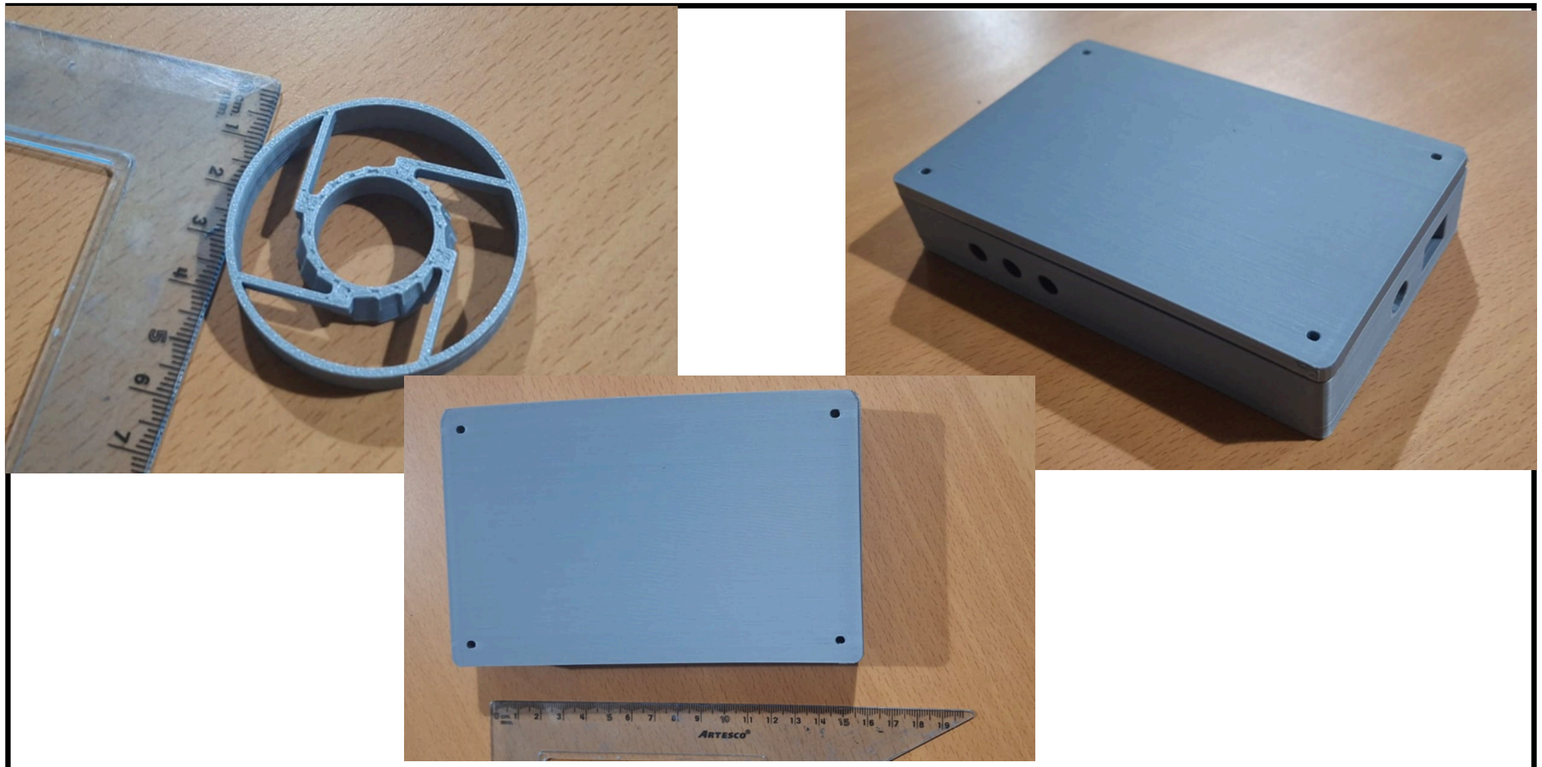
Comienzo del codigo: Aqui esta todo lo q va a pasar al incender el dispositivo, sirve para inicializar y calibrar todo el hardware antes de que el programa principal comience a funcionar de cada cosa, como la del MPU6050, HX711 y preparar el servomotor.

Programación y control

```
128  void loop() {
129    const uint32_t ahora = millis();
130    if (ahora - ultimaLecturaMs < INTERVALO_LECTURA_MS) return;
131
132    float anguloRelativo = 0.0f;
133    const bool imuOk = leerAngulo(anguloRelativo);
134    const bool hxOk = bascula.is_ready();
135    float fuerza = NAN;
136    if (hxOk) fuerza = bascula.get_units(3);
137
138    if (hxOk) {
139      if (!hayCarga && fuerza >= UMBRAL_CARGA) hayCarga = true;
140      if (hayCarga && fuerza <= UMBRAL_CARGA - HISTERESIS_CARGA) hayCarga = false;
141    }
142
143    // Reproduce la decisión del flujograma: apoyo detectado y ángulo menor que 0°.
144    if (sistemaActivo && hayCarga && hxOk && imuOk &&
145        anguloRelativo < ANGULO_DISPARO_GRADOS) {
146      const float objetivo = constrain(ANGULO_OBJETIVO_GRADOS,
147                                       ANGULO_MINIMO_OBJETIVO_GRADOS,
148                                       ANGULO_MAXIMO_GRADOS);
149      const float servoObjetivo = constrain(
150          SERVO_CENTRO_GRADOS + objetivo * SERVO_GRADOS_POR_GRADO_RODILLA,
151          SERVO_MINIMO_GRADOS, SERVO_MAXIMO_GRADOS);
152      anguloServoActual = servoObjetivo;
153      servoRodilla.write((int)anguloServoActual);
154    } else if (!hayCarga && sistemaActivo && hxOk) {
155      // Sin apoyo, vuelve al centro para dejar listo el siguiente ciclo.
156      anguloServoActual = SERVO_CENTRO_GRADOS;
157      servoRodilla.write((int)anguloServoActual);
158    }
159
160    if (ahora - ultimoEstadoMs >= INTERVALO_ESTADO_MS) {
161      ultimoEstadoMs = ahora;
162      Serial.printf("modo=%s | HX711=%s | carga=%.2f | apoyo=%s | IMU=%s | angulo_rel=%.1f deg | servo=%d deg\n",
163                    sistemaActivo ? "ACTIVO" : "REPOSO",
164                    hxOk ? "OK" : "ERROR", fuerza,
165                    hayCarga ? "SI" : "NO", imuOk ? "OK" : "ERROR",
166                    anguloRelativo, (int)anguloServoActual);
167    }
168  }
```

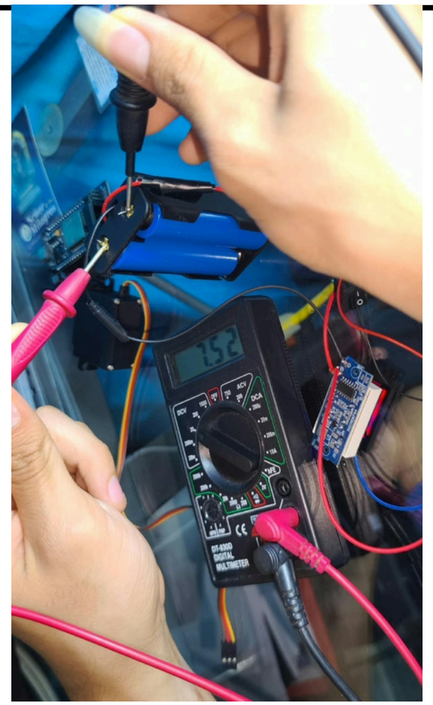
Codigo loop: El programa repite continuamente una secuencia de lecturas. Primero mide la fuerza de la pisada y la inclinación de la rodilla. Si detecta apoyo y el ángulo medido es menor que 0°, envía una señal al servomotor para ajustar la posición. Cuando deja de detectar apoyo, el servomotor vuelve a su posición central, listo para el siguiente paso.

Mecánica y fabricación



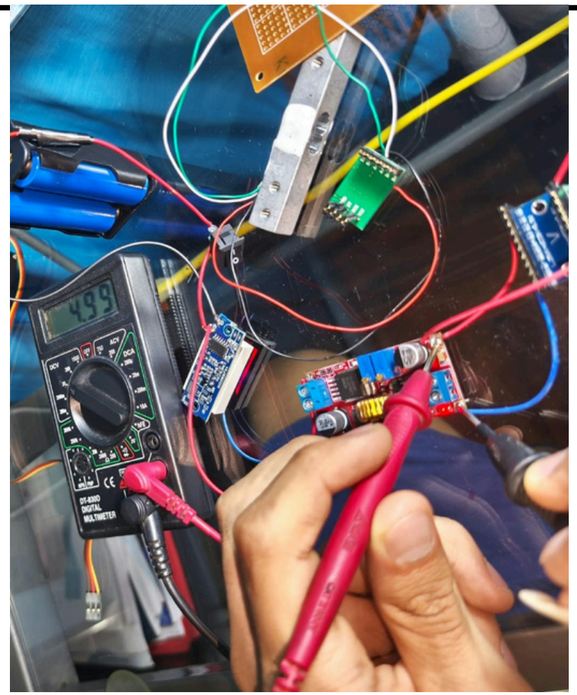
Se realizaron mediciones de las piezas impresas en 3D para verificar sus dimensiones y compararlas con los modelos CAD. Asimismo, se realizó el ensamblaje de la tapa con la caja de control para comprobar su ajuste y la alineación de los agujeros de fijación.

Electrónica y sensores



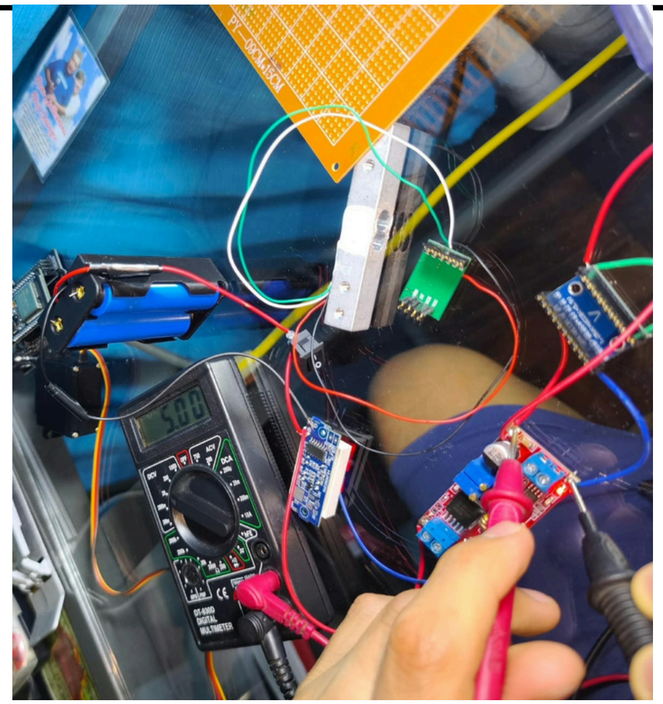
Medición de las Pilas (7.52V)

Electrónica y sensores



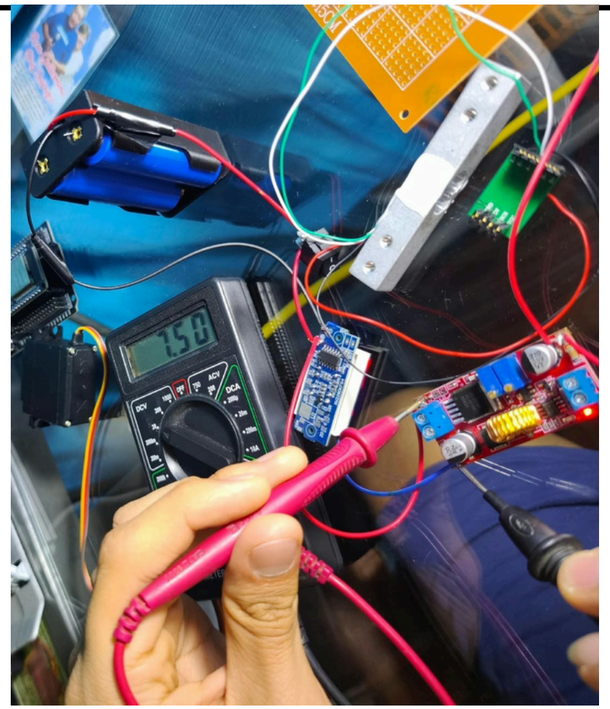
Medición regulador de voltaje 7.52V A 5.0 V (Salida)

Electrónica y sensores



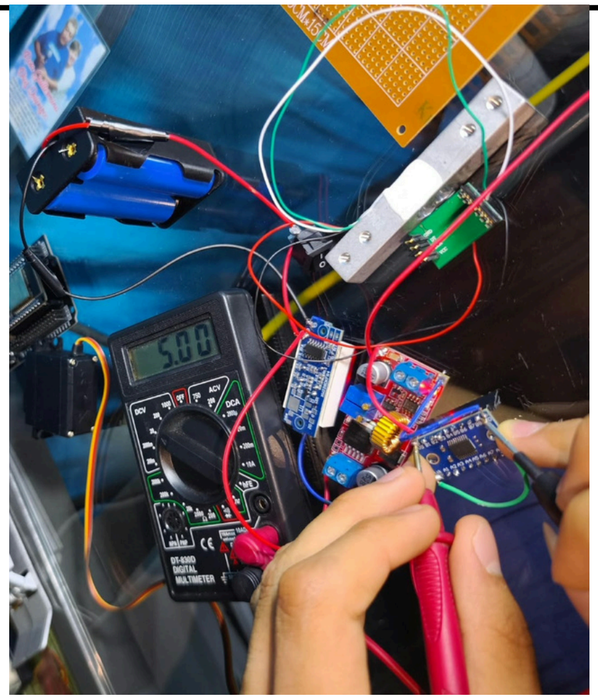
Medición regulador de voltaje (7.52V A 5.0 V)

Electrónica y sensores



Medición entrada 7.5V

Electrónica y sensores



Medición regulador 5V a 3.3V

EMPRESA BORCELLE

MUCHAS

GRACIAS

www.unsitiogeinal.es